

Hacettepe University
Computer Engineering Department
BBM442 Parallel Processing

MPI_Gather Simulation on 2D Torus Topology

Performance Comparison of Naive and Topology-Aware Gather Algorithms
using OMNeT++

Selected Algorithm: MPI_Gather

Selected Topology: 2D Torus

Simulator: OMNeT++

Baseline: Naive Gather

Optimized Method: Topology-aware row-level aggregation

Prepared by: Hüseyin Tenlik

Source Code Available At:

<https://github.com/htenlik/MPIGatherTorus/>

May 18, 2026

Contents

1	Introduction	2
2	Background	2
2.1	MPI_Gather Operation	2
2.2	2D Torus Topology	2
3	Implemented Algorithms	3
3.1	Naive Gather	3
3.2	Optimized / Topology-Aware Gather	3
3.2.1	Phase 1: Row-Level Gather	4
3.2.2	Phase 2: Column-Level Gather	4
4	OMNeT++ Simulation Design	4
4.1	Project Components	4
4.2	Routing Logic	5
4.3	Simulation Configurations	5
5	Simulation Parameters	5
6	Performance Metrics	5
6.1	Gather Completion Time	5
6.2	Total Hop Transmissions	6
6.3	Average Packet Latency at Root	6
6.4	Root Received Packets	6
6.5	Root Received Payloads	6
7	Simulation Results	6
7.1	Raw Results Table	6
7.2	Correctness Check	6
8	Performance Comparison	7
8.1	Completion Time	7
8.2	Total Hop Transmissions	7
8.3	Root Received Packets	8
8.4	Average Packet Latency at Root	8
8.5	Improvement Percentages	8
9	Discussion	9
9.1	Network Traffic	9
9.2	Root Load	9
9.3	Completion Time	10
9.4	Scalability Observation	10
10	Conclusion	10

1 Introduction

This project simulates the **MPI_Gather** collective communication operation on a **2D Torus** network topology using **OMNeT++**. The main objective is to compare a simple baseline gather algorithm with a topology-aware optimized gather algorithm.

In the assignment, the selected algorithm and topology are expected to be simulated in OMNeT++, and the performance comparison must be made against a naive approach. Therefore, this report compares:

- **Naive Gather:** every non-root node sends its data directly to the root using shortest-path routing.
- **Optimized / Topology-aware Gather:** nodes first gather data at row leaders, then row leaders send aggregated data to the root.

The experiments are performed on three different torus sizes: **4x4**, **8x8**, and **16x16**. The comparison focuses on completion time, total hop transmissions, average packet latency, number of packets received by the root, and correctness of the gathered payloads.

Source Code

The complete source code of the project is publicly available at:

<https://github.com/htenlik/MPIGatherTorus/>

2 Background

2.1 MPI_Gather Operation

MPI_Gather is a collective communication operation. In a gather operation, every participating process contributes its local data to a selected root process. At the end of the operation, the root process holds the data collected from all other processes.

For a system with P processes, where process 0 is selected as the root, the expected result is:

Root receives $P - 1$ payloads from non-root processes.

In this project, root rank is always selected as **rank 0**. Therefore, for each topology:

- 4x4 Torus: $16 - 1 = 15$ payloads must be received by the root.
- 8x8 Torus: $64 - 1 = 63$ payloads must be received by the root.
- 16x16 Torus: $256 - 1 = 255$ payloads must be received by the root.

The simulation results confirm that the root receives the correct number of payloads in all optimized and naive runs.

2.2 2D Torus Topology

A 2D Torus is a two-dimensional grid topology with wrap-around links. Unlike a simple 2D mesh, the boundary nodes are also connected:

- The leftmost and rightmost nodes of each row are connected.
- The top and bottom nodes of each column are connected.

This wrap-around structure reduces the maximum distance between nodes compared to a normal mesh. It also provides more routing flexibility because packets can move in either direction around the torus.

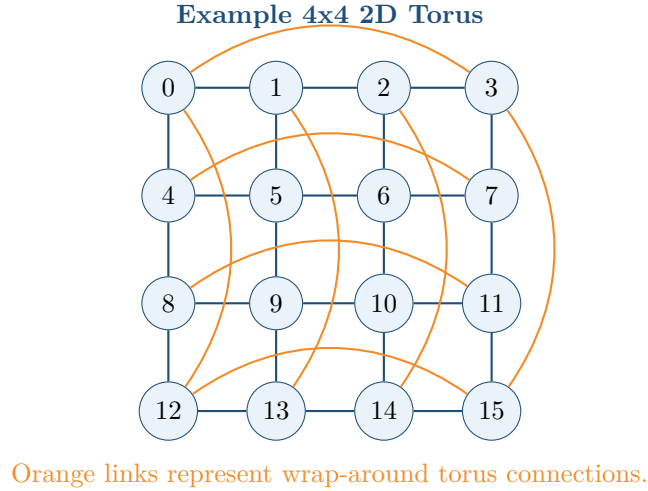


Figure 1: Illustration of a 4x4 2D Torus topology.

3 Implemented Algorithms

3.1 Naive Gather

The naive gather algorithm is used as the baseline. In this method, every non-root node sends one packet directly toward the root node. The packet is routed using shortest-path routing on the 2D Torus topology.

Naive Gather Characteristics

- Simple to implement.
- Every non-root node sends a separate packet to the root.
- The root receives $P - 1$ packets.
- Traffic becomes concentrated around the root.
- Root-side bottleneck becomes more visible as the network size increases.

The high-level behavior is:

```

1 for each node:
2     if node is not root:
3         create gather packet
4         route packet directly toward root using shortest path

```

For example, in a 16x16 topology with 256 nodes, the root receives 255 separate packets in the naive approach.

3.2 Optimized / Topology-Aware Gather

The optimized gather algorithm uses the structure of the 2D Torus. Instead of sending every packet independently to the root, the operation is divided into two phases.

3.2.1 Phase 1: Row-Level Gather

Each row first gathers its data at a **row leader**. In this project, the row leader is the node in the same column as the root.

For root rank 0, the root is located in column 0. Therefore, each row leader is the node at column 0 of its row.

3.2.2 Phase 2: Column-Level Gather

After a row leader collects all payloads from its row, it sends one aggregated packet to the root. This reduces the number of packets that arrive at the root.

Optimized Gather Characteristics

- Uses topology information.
- Reduces the number of packets received by the root.
- Reduces total hop transmissions.
- Introduces aggregation delay because row leaders must wait for row data.
- Aggregated packets may be larger than individual packets.

The high-level behavior is:

```

1 Phase 1: Row-level gather
2 for each row:
3     nodes in the row send their payloads to the row leader
4
5 Phase 2: Column-level gather
6 for each row leader:
7     row leader sends one aggregated packet to the root

```

4 OMNeT++ Simulation Design

4.1 Project Components

The implementation consists of OMNeT++ network description files, C++ module behavior, configuration files, and message definitions.

Table 1: Main project files and responsibilities.

File	Purpose
TorusNetwork.ned	Defines the 2D Torus network topology and node interconnections.
Node.ned	Defines the node module, gates, and parameters.
Node.cc	Implements node behavior, routing, naive gather, and optimized gather logic.
GatherMessage.msg	Defines the message structure used for gather packets.
omnetpp.ini	Contains simulation configurations for different topologies and algorithms.
results.txt	Stores measured simulation outputs.

4.2 Routing Logic

The routing uses shortest-path movement on the 2D Torus. Since the topology has wrap-around links, the shortest direction may be either the normal direction or the wrap-around direction.

For a torus dimension D , the circular distance between coordinates a and b is:

$$d(a, b) = \min(|a - b|, D - |a - b|)$$

This allows packets to avoid unnecessarily long paths around the grid.

4.3 Simulation Configurations

The following simulation configurations are used:

- Naive4x4
- Optimized4x4
- Naive8x8
- Optimized8x8
- Naive16x16
- Optimized16x16

5 Simulation Parameters

The same basic communication parameters are used for all experiments.

Table 2: Simulation parameters.

Parameter	Value
Simulator	OMNeT++
Selected collective operation	MPI_Gather
Selected topology	2D Torus
Root rank	0
Payload size	1024 bytes
Link delay	1 ms
Link datarate	10 Mbps
Start time	1 s
Tested topology sizes	4x4, 8x8, 16x16
Compared algorithms	Naive Gather, Optimized Gather

6 Performance Metrics

The following metrics are used to compare the algorithms.

6.1 Gather Completion Time

Gather completion time is the time at which the root has received all expected payloads. This is the main latency-oriented metric.

$$T_{\text{completion}} = T_{\text{last received payload at root}}$$

6.2 Total Hop Transmissions

Total hop transmissions represent the total number of link-level forwarding operations performed by packets. A lower value indicates less network traffic.

$$H_{\text{total}} = \sum_{\text{all packets}} \text{number of hops traveled}$$

6.3 Average Packet Latency at Root

Average packet latency measures the average time taken by packets that arrive at the root.

6.4 Root Received Packets

This metric counts the number of packet arrivals at the root. It is important because root overload is a major weakness of naive collective communication.

6.5 Root Received Payloads

This metric verifies correctness. The root must receive all payloads from non-root nodes.

$$\text{Expected payloads} = P - 1$$

7 Simulation Results

7.1 Raw Results Table

Table 3: Simulation results for naive and optimized gather.

Topology	Algorithm	Nodes	Completion Time (s)	Total Hop Transmissions	Average Latency (s)	Root Received Packets
4x4	Naive	16	1.0197344	32	0.00402709	15
4x4	Optimized	16	1.0251920	20	0.00456400	6
8x8	Naive	64	1.0698528	256	0.00816543	63
8x8	Optimized	64	1.0931520	144	0.01332910	14
16x16	Naive	256	1.2660112	2048	0.01621830	255
16x16	Optimized	256	1.3551616	1088	0.04444310	30

7.2 Correctness Check

The root received payload count confirms that the gather operation completed correctly in all tested topologies.

Table 4: Correctness verification using root received payloads.

Topology	Nodes	Expected Payloads	Received Payloads
4x4	16	15	15
8x8	64	63	63
16x16	256	255	255

Therefore, both algorithms produce correct gather results.

8 Performance Comparison

8.1 Completion Time

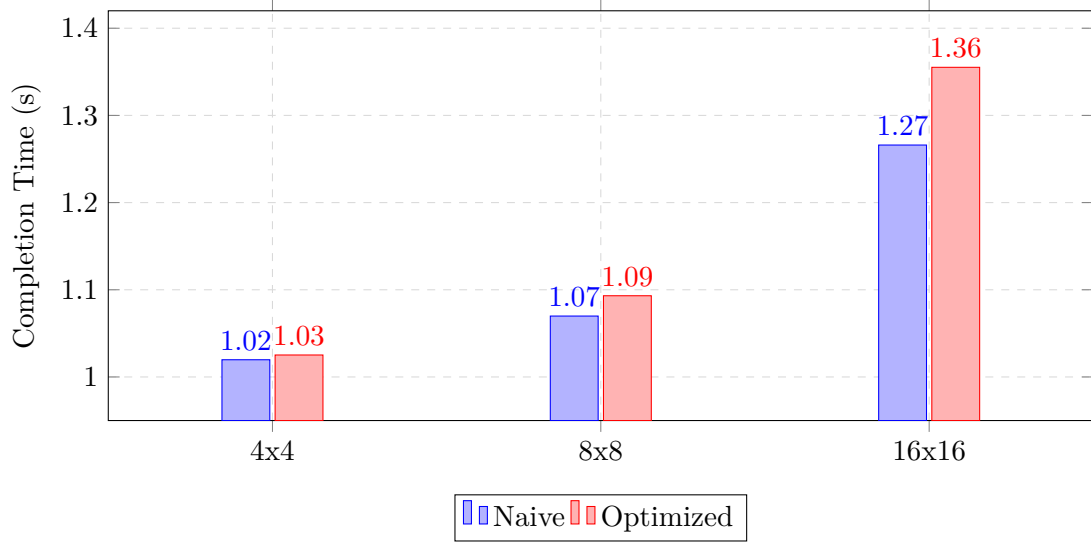


Figure 2: Gather completion time comparison.

The optimized algorithm has slightly higher completion time in all tested cases. This happens because row leaders first wait for row-level aggregation before forwarding aggregated packets to the root. Therefore, the optimized algorithm should not be interpreted as a pure latency optimization.

8.2 Total Hop Transmissions

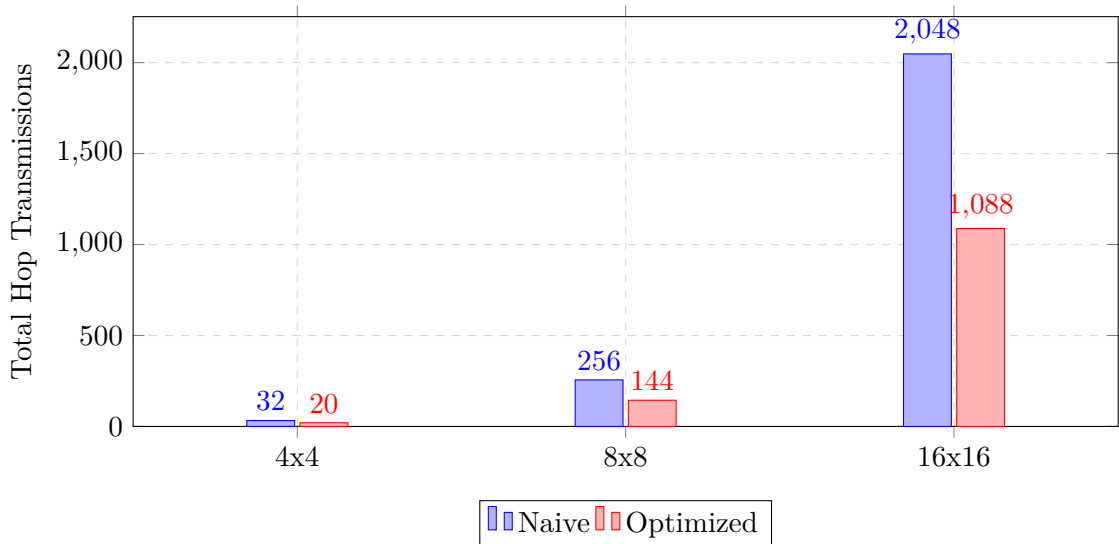


Figure 3: Total hop transmissions comparison.

The optimized algorithm significantly reduces total hop transmissions. This means fewer packet forwarding operations are performed in the network. The reduction becomes more important as the topology size increases.

8.3 Root Received Packets

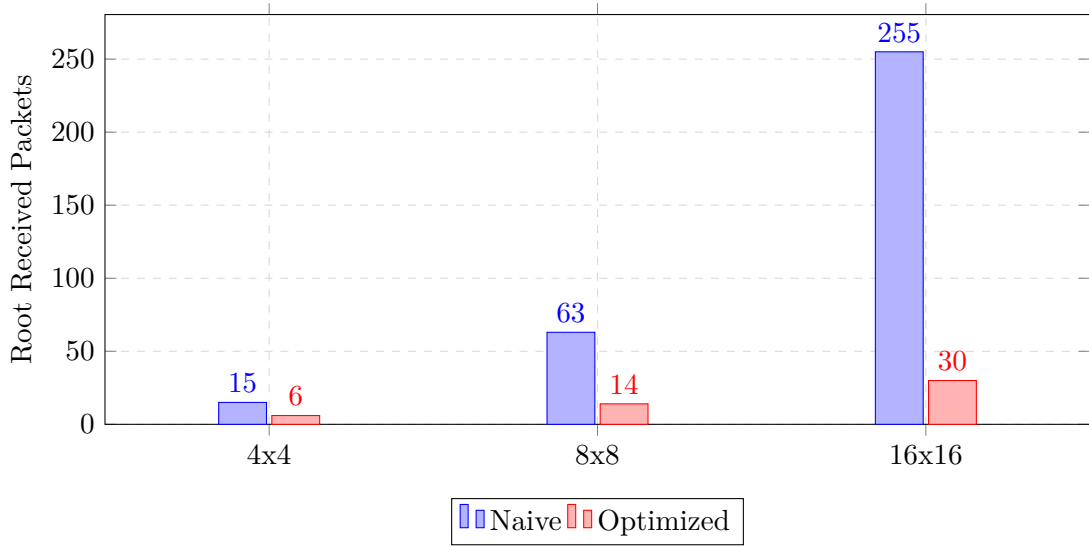


Figure 4: Number of packets received by the root.

The optimized method greatly decreases the number of packets received by the root. This is one of the most important advantages of the topology-aware approach because the root is the natural bottleneck in gather operations.

8.4 Average Packet Latency at Root

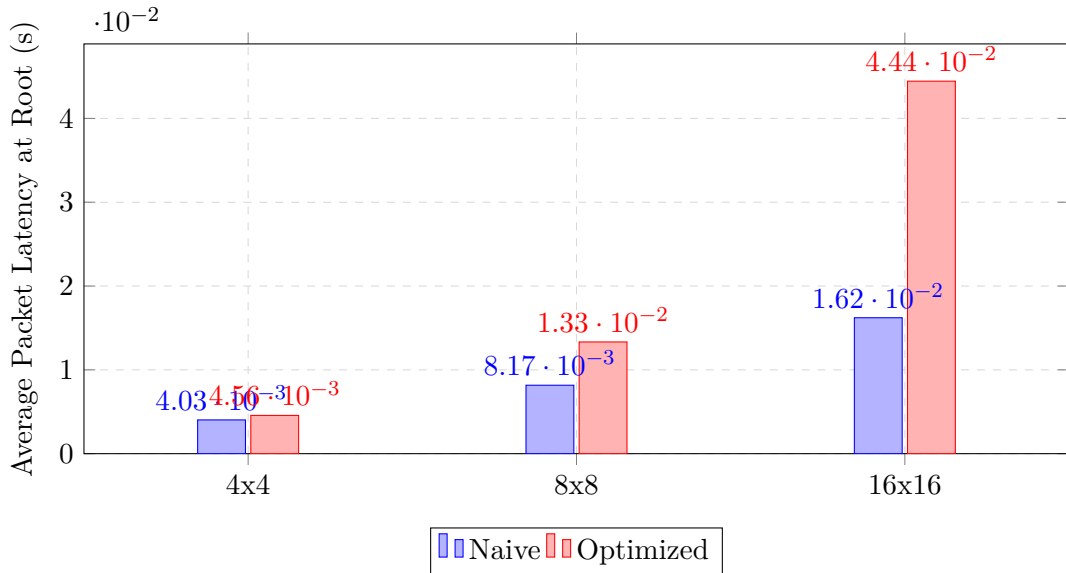


Figure 5: Average packet latency at the root.

Average packet latency is higher in the optimized method. This is expected because packets that reach the root in the optimized method are aggregated packets. Row leaders wait for data from their rows before sending the aggregated message to the root.

8.5 Improvement Percentages

The percentage reduction is calculated as:

$$\text{Reduction} = \frac{\text{Naive value} - \text{Optimized value}}{\text{Naive value}} \times 100$$

Table 5: Optimization effects compared to naive gather.

Topology	Hop Reduction	Root Packet Reduction	Completion Time Increase
4x4	37.50%	60.00%	0.54%
8x8	43.75%	77.78%	2.18%
16x16	46.88%	88.24%	7.04%

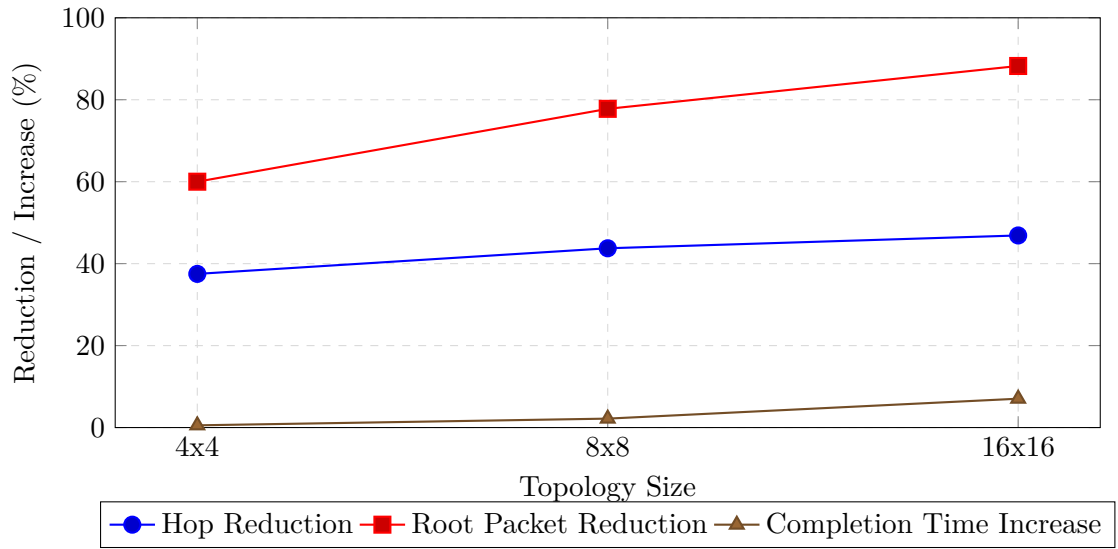


Figure 6: Percentage comparison of optimized gather against naive gather.

9 Discussion

The simulation results show a clear trade-off between network traffic and completion time.

9.1 Network Traffic

The optimized algorithm reduces total hop transmissions from:

- 32 to 20 in 4x4 Torus,
- 256 to 144 in 8x8 Torus,
- 2048 to 1088 in 16x16 Torus.

This means that the topology-aware gather method uses the network more efficiently. Instead of allowing every node to send a separate packet all the way to the root, packets are combined at row leaders.

9.2 Root Load

The root load reduction is even more significant. The root received packets decrease from:

- 15 to 6 in 4x4 Torus,

- 63 to 14 in 8x8 Torus,
- 255 to 30 in 16x16 Torus.

This is important because the root node can become a bottleneck in gather operations. The optimized approach reduces this bottleneck by forwarding aggregated packets instead of many individual packets.

9.3 Completion Time

The optimized algorithm has higher completion time:

- 0.54% increase in 4x4,
- 2.18% increase in 8x8,
- 7.04% increase in 16x16.

This increase is caused by two main reasons:

1. **Aggregation dependency:** row leaders must wait until they receive row data.
2. **Larger aggregated packets:** row leaders send combined data packets, which may take longer to transmit.

Therefore, the optimized method mainly improves **network traffic** and **root-side load**, not necessarily the final completion time.

9.4 Scalability Observation

As the topology size increases, the root packet reduction becomes stronger:

$$60.00\% \rightarrow 77.78\% \rightarrow 88.24\%$$

This shows that the topology-aware method becomes more beneficial for larger torus networks when root bottleneck and network traffic are more important concerns.

10 Conclusion

In this project, MPI_Gather was simulated on a 2D Torus topology using OMNeT++. Two algorithms were implemented and compared: a naive gather algorithm and a topology-aware optimized gather algorithm.

The simulation results show that the optimized gather algorithm correctly completes the gather operation for all tested topologies. The root receives all expected payloads in 4x4, 8x8, and 16x16 torus networks.

The optimized algorithm significantly reduces total hop transmissions and root received packets. For the largest tested topology, 16x16 Torus, total hop transmissions decreased from 2048 to 1088, and root received packets decreased from 255 to 30. This demonstrates that the topology-aware gather algorithm reduces network traffic and root-side bottleneck.

However, the optimized algorithm also increases completion time. This is due to row-level aggregation delay and larger aggregated packets. Therefore, the optimized approach is more suitable when the main goal is to reduce network traffic and root load rather than minimizing only completion time.

Overall, the topology-aware MPI_Gather approach is more scalable in terms of network usage and root pressure. It provides a meaningful optimization over the naive approach, especially as the torus size increases.

Appendix A: Repository Link

Source Code Available At:

<https://github.com/htenlik/MPIGatherTorus/>

Appendix B: Summary of Main Findings

- The selected algorithm is **MPI_Gather**.
- The selected topology is **2D Torus**.
- The simulator is **OMNeT++**.
- The naive algorithm sends every non-root packet directly to the root.
- The optimized algorithm first gathers data at row leaders, then sends aggregated packets to the root.
- The optimized algorithm reduces total hop transmissions.
- The optimized algorithm greatly reduces the number of packets received by the root.
- The optimized algorithm slightly increases completion time due to aggregation delay.
- The optimization becomes more meaningful as topology size increases.